Predicting Bridge Damage under Earthquakes

UE24CS352A Machine Learning mini project, Problem #27, Section C, Team 3
Guru R Bharadwaj (PES1UG24CS177), Chukkapalli Rohan (PES1UG24CS135)

Problem statement

Given a bridge and an earthquake, we want to say whether the bridge was damaged. The inputs are the age of the bridge, the magnitude of the earthquake and the distance from the bridge to the epicentre. The project follows Jiang and Zhang, *Predicting Bridge Performance under Earthquakes with Supervised Learning* (Stanford CS229, 2018), which compared Logistic Regression, QDA and KNN on this task. We reproduced their setup, fixed the parts of the evaluation we did not trust, and added two extensions: using material and bridge type as extra features, and predicting the level of damage.

Dataset

We used the authors' own data (`ds2.csv` from their GitHub repository). It has 249 bridge and earthquake pairs, of which only 21 (8.4%) are damaged. Damaged records come from government reports on well known earthquakes. Undamaged records are other earthquakes from the USGS catalogue that happened near the same bridges, on the assumption that no damage report means no damage. Each row also has the material (144 steel, 105 concrete), the bridge type (beam/girder, truss, cable or arch) and a damage level, where 11 rows are moderate or severe and 10 are partial or full collapse. We checked our cleaned file against the original row by row and replaced the Roman numeral codes with readable names.

One thing stood out while checking the data. All 21 damaged rows come from just seven earthquakes (Alaska 1964, San Fernando 1971, Miyagi 1978, Loma Prieta 1989, Northridge 1994, Kobe 1995 and Tohoku 2011), and every row from those seven is labelled damaged. This shaped one of our experiments.

Approach

With 8% positives, a model that always answers "undamaged" scores about 92% accuracy, so accuracy alone says little. We report recall on the damaged class, precision, F1, balanced accuracy and ROC AUC as well. We first make a stratified 70/30 split and only then up-sample the damaged rows in the training set by bootstrap, to positive to negative ratios of 0.2, 0.5 and 1.0. Doing it the other way round would copy the same damaged rows into both sets. Features are standardised for Logistic Regression and KNN, since distance is in tens of miles and magnitude sits between 3 and 9. QDA gets a small regularisation (reg_param 0.1) because about 15 damaged rows are not enough for a stable covariance estimate. For KNN, k is chosen by 5 fold cross validation on F1. Every number is an average over 100 random splits.

A random row split still lets the model train on one Northridge row and be tested on another, which mostly checks whether it remembers magnitude 6.7. So we also ran a grouped split: in each run, two of the seven damaging earthquakes and 30% of the other earthquake labels go to the test set, and no event appears on both sides.

Implementation

The code is plain Python with scikit-learn. `src/data.py` loads the data, makes the splits and does the resampling, and `src/models.py` builds the three models and picks k. `train_evaluate.py` runs the binary experiment for the core or full feature set, `train_multiclass.py` predicts the damage level, and `grouped_split.py` runs the grouped test. Each script writes tables and plots to `results/`. The best model by F1 is saved, and `predict.py` loads it for the live demo, for example `python predict.py --age 53 --mag 6.9 --dist 60`, which gives a 62% chance of damage for the Bay Bridge in Loma Prieta. The full binary run takes a few minutes on a laptop.

Results

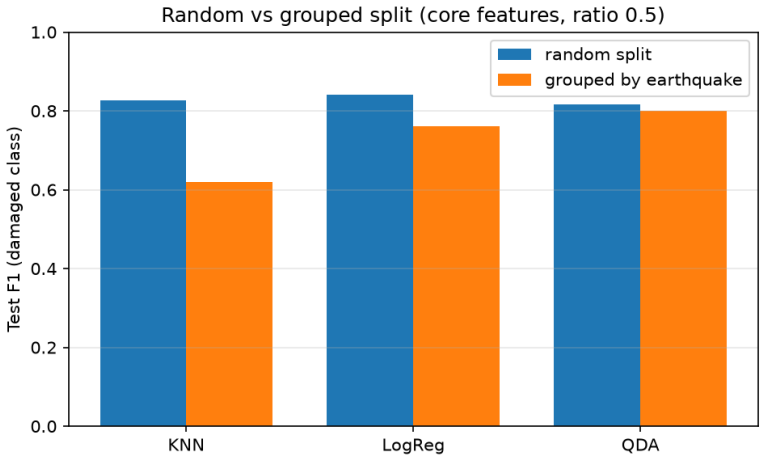
Model	Test acc	Balanced acc	Recall	Precision	F1	AUC
Logistic Regression	0.972	0.942	0.906	0.803	0.841	0.994
QDA	0.969	0.925	0.873	0.793	0.819	0.992
KNN	0.973	0.896	0.804	0.874	0.821	0.906

Table 1. Random split, core features, averaged over 100 runs and three resampling ratios.

On random splits Logistic Regression is slightly ahead on F1 and AUC. Raising the up-sampling ratio trades precision for recall: at ratio 1.0 Logistic Regression finds 95% of damaged bridges but only 73% of its alarms are correct. The reference report found 73.8% test accuracy for Logistic Regression and only 30% accuracy on the damaged class for KNN; scaling the features and picking k by F1 instead of accuracy is the likely reason for most of the KNN gap. Adding material and bridge type did not help. Logistic Regression stayed about the same (F1 0.849) and KNN fell to 0.672, probably because the binary columns distort distances.

Model	F1 random	F1 grouped	AUC random	AUC grouped
Logistic Regression	0.840	0.762	0.994	0.990
QDA	0.816	0.800	0.993	0.994
KNN	0.828	0.620	0.901	0.814

Table 2. Random split against the split grouped by earthquake (ratio 0.5, 100 runs).



Holding out whole earthquakes changes the ranking. KNN drops the most, from 0.83 to 0.62 F1, so part of its earlier score came from matching rows of the same event. Logistic Regression keeps its recall but raises more false alarms. QDA barely moves and becomes the best model, which matches the choice in the original report. For damage level, Logistic Regression reaches a macro F1 of 0.66, but with only 10 and 11 examples in the two damaged levels, recall on those levels is between 0.36 and 0.63.

Conclusions

Age, magnitude and distance are enough to separate damaged from undamaged bridges in this dataset, and with a fair evaluation QDA is the most dependable of the three models. The headline scores are higher than they would be in practice. The damaged rows come from seven famous large earthquakes, so magnitude alone does much of the work. The undamaged labels rest on the absence of a report, and bootstrap resampling repeats the same 21 positives without adding information. The grouped split is the number we would trust most, and even that rests on two held out events per run. More damaging events, and some undamaged bridges from those same events, would help this problem far more than a different model.